

Implementing Operator-Based Semantics for Normal Logic Programs using Meta-Programming in ASP: Appendix

Jesse Heyninck^{1,3} Spencer Killen²
Jeroen Spaans³

¹University of Cape Town, South Africa

²University of Alberta, Canada

³Open Universiteit, Heerlen, Limburg, the Netherlands

July 23, 2026

A Formal Description of Reification

For completeness, we give a more general, abstract formalization of the reification process. Because clingo performs some amount of program simplification before reification, obtaining the exact process is nontrivial. We define four functions: the injective functions $id_a : \mathcal{AP} \rightarrow \mathbb{N} \setminus \{0\}$, $id'_{lt} : \mathcal{AP} \rightarrow \mathbb{N}$, and general functions $id_{at}, id_{lt} : \mathcal{P} \rightarrow \mathbb{N}$ used to identify atoms, literal tuples (by atoms), atom tuples, and literal tuples (by rules) respectively. The ranges of these functions may overlap. Also, we assume that $id'_{lt}(b) = id_{lt}(p \leftarrow \psi)$ if $b = \psi$, and that id_{at} (resp. id_{lt}) produces the same value for any two rules with the same head (resp. body) and different values for any two rules with different heads (resp. bodies).

Given a program $\mathcal{P}'$ reified from $\mathcal{P}$ we have the transformation in Table 1. There is a literal tuple for every rule body and singular atom to tie it to the output. If an atom is deemed to be a tautology by clingo's simplifier, the tuple literal tying it to its output atom will be empty. The atom tuples correspond to the heads of rules and share atom identifiers with literal tuples ($id_a(b)$).

$literal_tuple(id_{lt}(p \leftarrow \psi)) \in \mathcal{P}'$ if $p \leftarrow \psi \in \mathcal{P}$
$literal_tuple(id_{lt}(p \leftarrow \psi), id_a(b)) \in \mathcal{P}'$ if $p \leftarrow \psi \in \mathcal{P}$ and $b \in \psi$
$literal_tuple(id_{lt}(p \leftarrow \psi), -id_a(b)) \in \mathcal{P}'$ if $p \leftarrow \psi \in \mathcal{P}$ and <i>not</i> $b \in \psi$
$literal_tuple(id'_{lt}(p)) \in \mathcal{P}'$ if $p \in \mathcal{A}_{\mathcal{P}}$
$literal_tuple(id'_{lt}(p), id_a(p)) \in \mathcal{P}'$ if $p \in \mathcal{A}_{\mathcal{P}}$
$atom_tuple(id_{at}(p \leftarrow \psi)) \in \mathcal{P}'$ if $p \leftarrow \psi \in \mathcal{P}$
$atom_tuple(id_{at}(p \leftarrow \psi), id_a(p)) \in \mathcal{P}'$ if $p \leftarrow \psi \in \mathcal{P}$
$rule(disjunction(id_{at}(p \leftarrow \psi)), normal(id_{lt}(p \leftarrow \psi))) \in \mathcal{P}'$ if $p \leftarrow \psi \in \mathcal{P}$
$output(p, id'_{lt}(p)) \in \mathcal{P}'$ where $p \in \mathcal{A}_{\mathcal{P}}$

Table 1: Formal descriptions of gringo’s reification process

B Supplementary Material on the Tool and Program Transformations

In this section, we provide more details on the tool, and the algorithms underlying the transformation component of our tool.

We’ve shown how to encode the Fitting operator $\mathcal{IC}_{\mathcal{P}}$ and ultimate operator $\mathcal{IC}_{\mathcal{P}}^u$ in various ways so that the fixpoints, and stable, Kripke-Kleene and well-founded fixpoints can be found. In this section, we present a tool that automatically defines these semantics for an arbitrary approximator (for normal logic programs) given as a logic program. A schematic representation of the processing pipeline of this tool is given in Figure 1. In more detail, the tool: (1) transforms an approximator program into a program that can be used to compute its Kripke-Kleene fixpoint, well-founded fixpoint, stable fixpoints and fixpoints; (2) Combines program reification and solving into a single command; (3) Deterministically provides output in a formatted, sorted and readable form.

The transformation component of our tool proceeds in two stages: first, an operator, inputed as a logic program, is transformed a new program where predicates have an additional ”step” parameter added to the end. This is necessary for some semantics, e.g. as we have seen, the stable semantics requires an internal and an external step. The transformation component is written in python, making use of clingo’s python api, in particular the library for its abstract syntax tree. In Appendix B, we give a formal

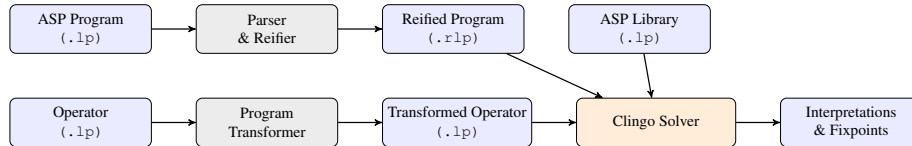


Figure 1: Processing pipeline for parsing, transforming, and solving ASP programs.

definition of the transformation, so that the interested reader might check whether their operator can be used in the tool. In slightly more detail, the transformation component first recursively analyzes the operator-program to determine which atoms require a step argument to be added. Then, given a chosen arity, the step arguments are added. Likewise, the program for computing a fixpoint is manipulated by the transformation component. Then, depending on which kind of fixpoint was selected, the necessary additional rules for the computation of that fixpoint are added.

Practically speaking, the tool supports several commands which can be listed by running `tool.py --help`. The primary ones are `well-founded`, `fixpoints`, `stable-fixpoints`, and `kripke-kleene`. Each command requires an approximator, which is provided as a file path, for example, `tool.py fixpoints fitting.lp` or `tool.py fixpoints ultimate.lp`. Such an invocation will generate and output a logic program for computing the given task with the given approximator. This program can be combined with a reified program to produce the appropriate answer set. For example, given an input logic program `input.lp`

```
tool.py fixpoints fitting.lp > program.lp
clingo --output=reify input.lp > input.rlp
clingo program.lp input.rlp
```

Running the above will generate the program that computes fixpoints of the approximator defined in `melvinfitting.lp` and store it in `program.lp`, then store the reified version of `input.lp` in the file `input.rlp`. Finally, the clingo solver will be invoked with both programs to compute the desired answer set. The above process can be simplified by passing a program to the tool (see below). Additionally, the tool will display data in a clean, formatted way. For example, given the following program named `program.lp` containing `a :- not b., b :- not a.,` running

```
tool.py stable-fixpoints fitting.lp --program program.lp
```

will result in the following output.

```
FIXPOINT ({ a }, { a })
FIXPOINT ({ b }, { b })
FIXPOINT ({ }, { a, b })
```

For the Kripke-Kleene and well-founded fixpoint, the different steps in the calculation are also displayed. There are also options to see the full answer set. The tool orders all answer set atoms based on their `model` and `step` arguments. The output tries to be deterministic in both the order of the atoms and the order of the answer sets

While we consider our algorithm to be fairly generic, it cannot possibly accommodate all possible approximators encoded as logic programs. So that users of our tool may determine whether an approximator is supported, we briefly outline our transformation algorithms as pseudocode.

First, we have an algorithm that recursively analyzes a program to determine which atoms require a step argument to be added. Intuitively, we begin with the `hold/2` atom, then locate any atoms which appear in rules with `hold/2` in the body.

```
targets ← [hold/2]
while  $\exists (p \leftarrow \psi) \in \mathcal{P}, \neg \text{targets.match}(p), \exists q \in \psi, \text{target.match}(\Pi)$  do
    targets ← targets + [nameof(p)/arityof(p)]
end while
```

The function `targets.match` determines whether a given atom matches the name and arity of a predicate in the list, `targets`. Naturally, `nameof` and `arityof` are functions that extract the name and arity of atoms, respectively. The result of running this algorithm on the Melvin-Fitting operator is the list `[hold/2, conjunction/2]`. Thus, for the tool to perform iterative computation using the input approximator a “step” argument must be added to both the hold and conjunction atoms. In the following, we give the algorithm to add step parameters for the purpose of computing fixpoints.

```

for  $t(V_0, V_1, \dots, V_i) \leftarrow \dots \in \mathcal{P}$  do
  if targets.match( $t(V_0, V_1, \dots, V_i)$ ) then
    Replace with  $t(V_0, V_1, \dots, V_i, \text{step}(I, K + 1))$ 
    Add  $\text{step}(I, K + 1)$  to the body of the rule.
  end if
end for
for  $\dots \leftarrow \dots, t(V_0, V_1, \dots, V_i), \dots \in \mathcal{P}$  do
  if targets.match( $t(V_0, V_1, \dots, V_i)$ ) then
    Replace with  $t(V_0, V_1, \dots, V_i, \text{step}(I, K))$ 
    Add  $\text{step}(I, K)$  to the body of the rule.
  end if
end for

```

An identical algorithm is used to construct the fixpoints operator, but instead of $\text{step}(I, K)$ and $\text{step}(I, K + 1)$, the atoms $\text{step}(I)$ and $\text{step}(I + 1)$ are used. Unlike the fixpoints operator, the construction of a stable operator requires more steps. Namely, references to data from the upper bound must come from the previous step when computing the lower bound (`t` and `h` are computed in isolation). The following algorithm determines whether a step argument needs to be decremented based on the model argument (either `h` or `t`) occurring in the head of the rule.

```

for  $t(V_0, V_1, \dots, V_i) \leftarrow \psi \in \mathcal{P}$  do
   $\text{model} \leftarrow \text{modelof}(t(V_0, V_1, \dots, V_i))$ 
  if model = undefined then
    continue
  end if
  for  $g(V_0, \dots, \text{step}(I, K), \dots, V_i) \in \psi$  do
    Replace  $\text{step}(I, K)$  with  $\text{step}(I - 1)$ 
  end for
end for

```

The function `modelof` gets the model from the model argument of a function (either `h`, `t`, or undefined if no such argument is present). This ensures that references that depend on the opposite model take data that was computed during the previous computation of stable revision.

C Correctness Proofs

In this appendix, we show the correctness of our encodings. We assume an arbitrary but fixed ground normal logic program $\mathcal{P}$.

We denote $\text{reif}(h_i, t_i) = \{\text{hold}(a, m, i) \mid a \in m_i, m \in \{h, t\}\}$ and $\text{reif}(\mathcal{P})$ is the reification of $\mathcal{P}$. $\Pi_{\mathcal{IC}}$ is given in listing 3. We denote the program in Listing 4 by Π_{fp} , the program in Listing 5 by Π_{KK} , and the program in listing 6 by $\Pi_{S(\mathcal{IC})}$. Finally, $\Pi_{\mathcal{IC}^u}$ is given in listing 7.

Theorem 1. *For any $h_i \subseteq t_i \subseteq \mathcal{A}$, $\Pi_{\mathcal{IC}} \cup \text{reif}(h_i, t_i) \cup \text{reif}(\mathcal{P})$:*

1. *has a unique answer set, and*
2. *$a \in \mathcal{IC}^l(h_i, t_i)$ iff $\text{hold}(a, h, i+1)$ is in this answer set (and similarly for $\mathcal{IC}^u$).*

Proof. That $\Pi_{\mathcal{IC}_{\mathcal{P}}} \cup \text{reif}(h_i, t_i)$ has a unique answer set is immediate from the fact that $\Pi_{\mathcal{IC}_{\mathcal{P}}} \cup \text{reif}(h_i, t_i)$ is a hierarchical disjunctive logic program where every disjunction (line 3 of listing 3) contains a unique $\text{atom_tuple}(H, A)$, i.e. is actually normal (since $\mathcal{P}$ is normal). Suppose now that $a \in \mathcal{IC}_{\mathcal{P}}^l(h_i, t_i)$, i.e. there is some $a: -a_1, \dots, a_m, \text{not } a_{m+1}, \dots, \text{not } a_n$ s.t. $a_j \in h_i$ for every $j = 1 \dots m$ and $a_j \notin t_i$ for $j = m+1 \dots n$. Then because of the form of $\text{reif}(h_i, t_i)$ and $\text{reif}(\mathcal{P})$, and in view of line 1 of Listing 3, $\text{conjunction}(a, h, i)$ is in the answer set. Again because of the form of $\text{reif}(\mathcal{P})$ (and because $\mathcal{P}$ is normal, i.e. $\text{atom_tuple}(H, A)$ is unique for every H) and using line 3 of listing 3, we derive $\text{hold}(a, t, i+1)$. The other direction is similar. $\square$

Theorem 2. *If s is an answer set of $\Pi_{\text{fp}} \cup \Pi_{\mathcal{IC}} \cup \text{reif}(\mathcal{P})$ and $x = \{a \mid \text{hold}(a, x, 0) \in s\}$ for $x \in \{h, t\}$, then $(h, t) = \mathcal{IC}_{\mathcal{P}}(h, t)$. Similarly, given some $h \subseteq t \subseteq \mathcal{A}_{\mathcal{P}}$ and $(h, t) = \mathcal{IC}_{\mathcal{P}}(h, t)$, there exists an answer set s of $\Pi_{\mathcal{IC}} \cup \text{reif}(\mathcal{P}) \cup \Pi_{\text{fp}}$ s.t. $x = \{a \mid \text{hold}(a, x, 0) \in s\}$ for $x \in \{h, t\}$.*

Proof. Suppose s is an answer set of $\Pi_{\mathcal{IC}} \cup \text{reif}(\mathcal{P}) \cup \Pi_{\text{fp}}$. Let $h = \{a \mid \text{hold}(a, h, 0) \in s\}$ and $t = \{a \mid \text{hold}(a, t, 0) \in s\}$. Notice that atoms of the form $\text{hold}(a, x, 0)$ with $x \in \{h, t\}$ only occur in the head of one rule, namely the choice rule on line 1 of Listing 4. We first show that $h \subseteq \mathcal{IC}_{\mathcal{P}}^l(h, t)$. Suppose $a \in h$. In view of the constraint on line 6, and the rules on line 4, 5 and 3, we can derive that $\text{hold}(a, h, 0) \in s$ (as $a \in h$) implies $\text{hold}(a, h, 1) \in s$. Inspection of $\Pi_{\mathcal{IC}} \cup \text{reif}(\mathcal{P}) \cup \Pi_{\text{fp}}$ reveals that atoms with $\text{hold}(a, h, 1)$ in the head only occur in $\Pi_{\mathcal{IC}}$. Similarly to Theorem 1, we can show that this means that $a \in \mathcal{IC}_{\mathcal{P}}^l(h, t)$. Showing that $h \supseteq \mathcal{IC}_{\mathcal{P}}^l(h, t)$ and $t = \mathcal{IC}_{\mathcal{P}}^u(h, t)$ is done in a similar way. $\square$

Theorem 3. *For any $h_i \subseteq t_i \subseteq \mathcal{A}$:*

1. *$\Pi_{S(\mathcal{IC})} \cup \text{reif}(h_i, t_i) \cup \text{reif}(\mathcal{P})$ has a unique answer set s , and*
2. *$a \in S(\mathcal{IC}^l)(h_i, t_i)$ iff $\text{hold}(a, t, i+1) \in s$ (and similarly for $S(\mathcal{IC}^u)$).*

Proof. We first show that for any answer set s of $\text{reif}(h_i, t_i) \cup \text{reif}(\mathcal{P}) (\dagger)$: $(\mathcal{IC}_{\mathcal{P}}^l)^i(\emptyset, t) = \{a \mid \text{hold}(a, h, \text{step}(0, i)) \in s\}$ for any $i \geq 0$ (where $t = \{a \mid \text{hold}(a, t, \text{step}(-1)) \in s\}$). For the base case ($i = 1$), notice that there is no rule with $\text{hold}(a, h, \text{step}(0, 0))$ in the head, and so $\text{conjunction}(a, h, \text{step}(0, 0))$ will only be derivable for rules with no positive body atoms (line 1 of Listing 5). Thus, with line 4 of Listing

5, we see that $\text{hold}(a, h, \text{step}(0, 1))$ is in the answer set only if there is a rule $a \leftarrow \neg b_1, \dots, \neg b_m$, where $b_1, \dots, b_m \notin t$, which establishes $\mathcal{IC}_{\mathcal{P}}^l(\emptyset, t) = \{a \mid \text{hold}(a, h, \text{step}(0, 1)) \in s\}$. The inductive case is similar.

We now show that $\Pi_{S(\mathcal{IC})} \cup \text{reif}(h_i, t_i) \cup \text{reif}(\mathcal{P})$ has a unique answer set s . Indeed, as $\Pi_{S(\mathcal{IC})} \cup \text{reif}(h_i, t_i) \cup \text{reif}(\mathcal{P})$ is a stratified normal logic program with one constraint, we see that the program has one or no answer sets. We have to show that the body of the constraint $:- \text{not } \text{fp}(\mathcal{M}), \text{model}(\mathcal{M})$ is never satisfied, i.e. that $\text{fp}(\mathcal{M}), \text{step}(S1, S2)$ is derivable. In fact, note that $\mathcal{IC}_{\mathcal{P}}^l(\emptyset, t)$ and $\mathcal{IC}_{\mathcal{P}}^u(h, \emptyset)$ are guaranteed to converge to a fixpoint after at most $|\mathcal{A}|$ iterations (as they are monotonic operators). With $\dagger$, and since we have $\text{step}(0, S)$ for $S = 0 \dots |\mathcal{A}|$ by line 1 of Listing 4, we see that $\text{fp}(\mathcal{M}, \text{step}(0, S))$ will be derivable for $\mathcal{M} = h, t$ and $S \leq |\mathcal{A}|$. $\square$

Theorem 4. *If s is an answer set of $\Pi_{S(\mathcal{IC})} \cup \Pi_{\text{fp}} \cup \text{reif}(\mathcal{P})$ and $x = \{a \mid \text{hold}(a, x, 0) \in s\}$ for $x \in \{h, t\}$, then $(h, t) = S(\mathcal{IC}_{\mathcal{P}})(h, t)$. Similarly, where $h \subseteq t \subseteq \mathcal{A}_{\mathcal{P}}$ and $(h, t) = S(\mathcal{IC}_{\mathcal{P}})(h, t)$, there exists an answer set s of $\Pi_{S(\mathcal{IC})} \cup \Pi_{\text{fp}} \cup \text{reif}(\mathcal{P})$ s.t. $x = \{a \mid \text{hold}(a, x, 0) \in s\}$ for $x \in \{h, t\}$.*

Proof. Similar to the proof of Theorem 2. $\square$

Theorem 5. $\Pi_{\mathcal{IC}} \cup \Pi_{\text{fp}} \cup \Pi_{\text{KK}} \cup \text{reif}(\mathcal{P})$ has a unique answer set, and where $x = \{a \mid \text{hold}(a, x, _) \in s\}$ for $x \in \{h, t\}$, (h, t) is the Kripke-Kleene model of $\mathcal{P}$. Likewise, $\Pi_{S(\mathcal{IC})} \cup \Pi_{\text{fp}} \cup \Pi_{\text{KK}} \cup \text{reif}(\mathcal{P})$ has a unique answer set, and where $x = \{a \mid \text{hold}(a, x, _) \in s\}$ for $x \in \{h, t\}$, (h, t) is the well-founded model of $\mathcal{P}$.

Proof. Similar to the above proofs, where can abstract away from the step (cf. $\text{hold}(a, x, _)$ in the theorem's statement) as the upper and lower bound of the Kripke-Kleene and well-founded model grow monotonically. $\square$

Theorem 6. *For any $h_i \subseteq t_i \subseteq \mathcal{A}$, it holds that for any answer set s of $\text{reif}(h_i, t_i) \cup \text{reif}(\mathcal{P}) \cup \Pi_{\mathcal{IC}^u}$, $a \in \mathcal{IC}^{u,l}(h_i, t_i)$ iff $\text{hold}(a, t, i+1)$ is in this answer set (and similarly for $\mathcal{IC}^{u,l}$).*

Proof. We first show that for any answer set s of $\text{reif}(h_i, t_i) \cup \text{reif}(\mathcal{P})$ ($\dagger$): $(\mathcal{IC}_{\mathcal{P}}^{u,l})(h_i, t_i) = \{a \mid \text{hold}(a, h, i) \in s\}$.

In fact, we first show that $\dagger$: $\text{bot}(a) \in s$ iff $[B_{\mathcal{P}}(a)]_{h_i, t_i}$ is unsatisfiable. We denote the DNF-formula $[B_{\mathcal{P}}(a)]_{h_i, t_i}$ by $\bigvee_{i=0}^n \Delta_i$

$\Rightarrow$: We proceed by showing the contrapositive. Suppose that $[B_{\mathcal{P}}(a)]_{h_i, t_i}$ is satisfiable. This means that there is a set $\Delta \subseteq \bigcup_{j=0}^n \Delta_j$ s.t. for every $j = 1 \dots n$, there is some $b \in \Delta_j$ s.t. $b \in \Delta$ or there is some $\neg b \in \Delta_j$ s.t. $a \notin \Delta$. It can then be shown that $\{\text{true}(|b|, a) \mid b \in \Delta\} \cup \{\text{false}(|\neg b|, a) \mid b \notin \Delta\}$ is a minimal model of the rules 7-15 of listing 7. Indeed, notice that by rule 7, any such model will contain at least $\text{true}(|b|, a)$ or $\text{false}(|\neg b|, a)$ for every $b \in \bigcup_{i=0}^n \Delta_i$. We can furthermore verify that the rules on 8 and 9 are satisfied, whereas the body of the rule on line 12 is not, so $\text{unsatNeg}(X, A)$ is not derivable for any X .

$\Leftarrow$: Suppose that $[B_{\mathcal{P}}(a)]_{h_i, t_i}$ is unsatisfiable. This means that for every $\Delta \subseteq \bigcup_{j=0}^n \Delta_j$ (where every j corresponds to a tuple T for which $\text{literal_tuple}(T, a)$ holds), and every $j = 1 \dots n$, $\Delta_j \subset \Delta$ or there is some $a \in \Delta_j$ s.t. $a \notin \Delta$. This means that for every possible disjoint assignment of $\text{true}(|b|, a)$ and $\text{false}(|b|, a)$ to the atoms $\{b \mid b \in \bigcup_{i=1}^n \Delta_i\}$, $\text{unsatNeg}(T, a)$ is derivable (where T corresponds to the relevant i) according to one of the rule on line 12. Then via the rule on line 13, we derive $\text{bot}(a)$.

That $(\mathcal{IC}_{\mathcal{P}}^{\mathcal{U}, l})(h_i, t_i) = \{a \mid \text{hold}(a, h, \text{step}(0)) \in s\}$ now follows from Theorem ?? and $\dagger$. $\square$

Correctness of the encodings of fixpoint, stable fixpoint, Kripke-Kleene and well-founded semantics of the ultimate operator is proven in an analogous way.

D Additional Figures

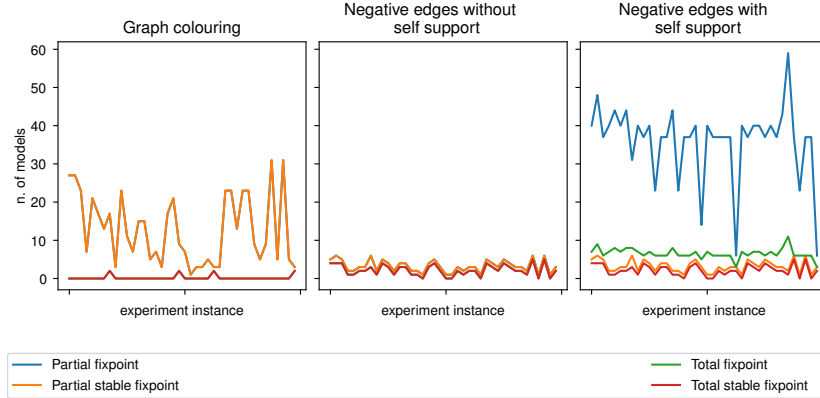


Figure 2: The number of models for each graph experiment and semantics, grouped by graph problem.

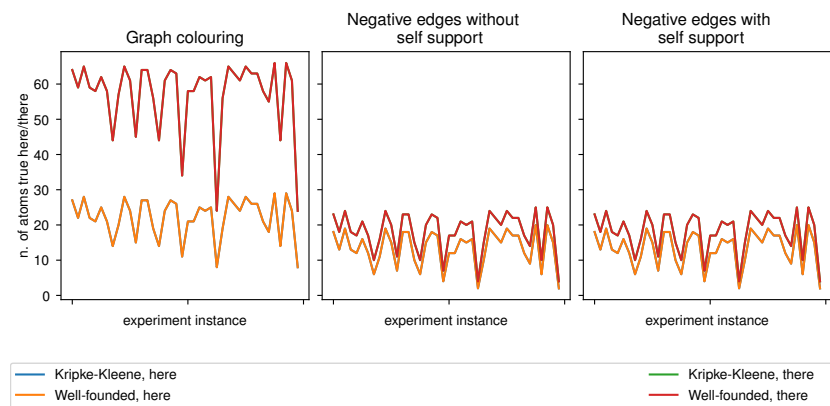


Figure 3: Precision of Kripke-Kleene and well-founded models, number of atoms true here and there for each graph experiment, grouped by graph problem.